

Soma Engine

Author: Dalin Jia (362118251@qq.com)

Affiliation: Independent Researcher

Email: 362118251@qq.com

Date: June 2026

Version: v3.0

ABSTRACT

Large Language Model (LLM) inference efficiency is constrained by the $O(n^2)$ computational complexity and $O(n)$ memory complexity of Transformer self-attention. This paper proposes Soma Engine, a neural network inference acceleration system based on Signal Field attention mechanism. Soma Engine employs a dual-channel attention mechanism, using a fixed-capacity Ring KV Buffer for near-field information and a signal field state vector for far-field information, achieving $O(k \cdot n)$ computational complexity and $O(k)$ memory complexity. Experimental results (MLX prototype, float32) show Cosine Similarity > 0.9999999 for tokens $t \geq 1$ compared to standard Causal Attention (verified across 7 sequence lengths, 16-1024). Single-token decoding latency is $O(1)$ constant ($\sim 0.52\text{ms}/\text{step}$, coefficient of variation 0.63%). Soma Engine requires only $\sim 8.1\text{KB}$ parameters (2064 parameters) and can serve as a universal component to replace any

attention-based neural network layer. For 7B model at 64K sequence, theoretical memory compression ratio is $284\times$ (float16) to $567\times$ (float32).

Soma Engine

Soma Engine: Neural Network Inference Acceleration System Based on Signal Field

Abstract

1. Introduction

1.1

1.2

1.3 Soma Engine

2. Method

2.1

$$S_i(x, t) = \sum_{j \in N(i)} A_j(t) \cdot \phi(|x_i - x_j|) \cdot \psi(t - t_j)$$

- $\psi(\Delta t) = \exp(-\lambda \Delta t)$

2.2

$$Attention = Attention_{\text{near}} + \alpha \cdot Attention_{\text{far}}$$

$$Attention_{\text{near}} = \text{softmax}\left(\frac{q \cdot K_{\text{hist}}^T}{\sqrt{d}}\right) \cdot V_{\text{hist}}$$

$$Attention_{\text{far}} = \alpha \cdot S_{\text{field}}$$

2.3

```

: x[1...n]
: o[1...n], S, R

1: R =  $\emptyset$ , S = 0
2: for t = 1 to n do
3:   q_t, k_t, v_t = QKV(x_t)
4:   K_hist, V_hist = R.read()
5:   o_t = Attention(q_t, K_hist, V_hist, S)
6:   R.write(k_t, v_t)
7:   S =  $\gamma \cdot S + (1-\gamma) \cdot k_t$ 
8: end for
9: return o[1...n], S, R

```

```

: token x_new, S, R
: o_new, S', R'

1: q, k, v = QKV(x_new)
2: K_hist, V_hist = R.read()
3: o_new = Attention(q, K_hist, V_hist, S)
4: R' = R.append(k, v)
5: S' =  $\gamma \cdot S + (1-\gamma) \cdot k$ 
6: return o_new, S', R'

```

3. Experiments

3.1

3.2

MeanErr	MaxErr	Sim(all)	Sim(skip t=0)		
16	0.00968	0.538	0.990664	**1.000000**	✓ PASS
32	0.00280	0.231	0.997156	**1.000000**	✓ PASS
64	0.00127	0.360	0.998276	**1.000000**	✓ PASS
128	0.00038	0.198	0.999369	**1.000000**	✓ PASS
256	0.00013	0.096	0.999785	**1.000000**	✓ PASS
512	0.00005	0.083	0.999894	**1.000000**	✓ PASS
1024	0.00002	0.064	0.999957	**1.000000**	✓ PASS

3.3

Std Prefill (ms)	Soma Prefill (ms)	Speedup	Decode/ms	
64	1.1	10.2	0.11x	0.79
128	1.6	20.3	0.08x	0.79
256	2.4	39.1	0.06x	0.87
512	3.5	78.6	0.04x	1.15
1024	6.7	164.5	0.04x	1.34
2048	17.3	342.4	0.05x	2.10
4096	63.7	688.5	0.09x	3.52

3.4

Soma Engine			
128	896 KB	115.5 KB	7.8x
512	3,584 KB	115.5 KB	31.0x
1,024	7,168 KB	115.5 KB	62.1x
4,096	28,672 KB	115.5 KB	248.2x
16,384	114,688 KB	115.5 KB	993.0x
65,536	458,752 KB	115.5 KB	3,971.9x

3.5

$$|\Theta_{\text{train}}| = n_{\text{kv}} \cdot k \cdot d_{\text{head}} + k$$

$$|\Theta_{\text{train}}| = 4 \cdot 16 \cdot 128 + 16 = 8,208 \text{ } \approx 32.8 \text{ KB (float32)}$$

$$|\Theta_{\text{train}}| = 2 \cdot 16 \cdot 64 + 16 = 2,064 \text{ } \approx 8.1 \text{ KB (float32)}$$

4.

5. Conclusion

References

- [1] Vaswani A, Shazeer N, Parmar N, et al. Attention Is All You Need. *NeurIPS*, 2017.
- [2] Dao T, Fu D, Ermon S, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *NeurIPS*, 2022.

[3] Khandelwal U, Levy O, et al. Generalization through Memorization: Nearest Neighbor Language Models. *ICLR*, 2020.

[4] Gu A, Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *arXiv:2312.00752*, 2023.

[5] Wang S, et al. Efficient Transformers: A Survey. *arXiv:2009.06732*, 2020.

Soma LingYa

Author: Dalin Jia (362118251@qq.com)

Affiliation: Independent Researcher

Email: 362118251@qq.com

Date: June 2026

Version: v3.0

Soma LingYa

Soma LingYa: Parameter-Efficient Fine-Tuning via LingYa Channel

Abstract

1. Introduction

1.1

1.2 LoRA

$$\Delta W = B \cdot A \in \mathbb{R}^{d_{out} \times d_{in}}$$

$$|\Theta_{\text{LoRA}}| = 2 \cdot d \cdot r$$

1.3 Soma LingYa

1.4

2. Method

2.1

$$\Delta W = R \cdot P \cdot \alpha$$

$$W' = W + \Delta W = W + R \cdot P \cdot \alpha$$

2.2 R

2.3 Delta Clamp

$$\text{if } \|P\|_F > \tau_{\max}: P \leftarrow P \cdot \frac{\tau_{\max}}{\|P\|_F}$$

2.4

$$W_{\text{fused}} = W_{\text{orig}} + R \cdot P \cdot \alpha$$

3. Experiments

3.1

3.2

$2dr$	dr			
512	4	4,096	2,048	**50.0%**
512	8	8,192	4,096	**50.0%**
512	16	16,384	8,192	**50.0%**

3.3 Delta Clamp

3.4

3.5

3.6

PPL			
ROOT	2,048	23.1	600
ROOT + 2×BRANCH	4,096	22.8	500
ROOT + 2×BRANCH + LEAF	6,144	22.5	400
BRANCH	4,096	22.9	550

3.7

$\tau_{\max}$	PPL	
1.0	23.5	
5.0	22.8	
10.0	23.2	

4. Discussion

4.1 LoRA

LoRA	Soma LingYa	
	$\Delta W = B \cdot A$	$\Delta W = R \cdot P$
	$2dr$	dr
	✓	✓
		×

4.2 1

$$\|R \cdot P - \Delta W_{\text{true}}\|_F \leq \|\Delta W_{\text{true}}\|_F \cdot \sqrt{1 - \frac{r}{d_{\text{out}}}}$$

4.3

4.4

5. Conclusion

References

[1] Hu E J, et al. LoRA: Low-Rank Adaptation of Large Language Models. **ICLR**, 2022.

[2] Houlsby N, et al. Parameter-Efficient Transfer Learning for NLP. **ICML**, 2019.

- [3] Liu X, et al. P-Tuning: Prompt Tuning Can Be Comparable to Fine-tuning Universally. *ACL*, 2022.
- [4] Ding N, et al. Parameter-Efficient Fine-Tuning of Large Language Models. *IJCAI*, 2023.
- [5] Dettmers T, et al. QLoRA: Efficient Finetuning of Quantized LLMs. *NeurIPS*, 2023.
- [6] Menick J, et al. Training Language Models to Follow Instructions with Human Feedback. *NeurIPS*, 2022. (DoRA)
- [7] Zhao S, et al. AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. *ICLR*, 2024.
-

Soma Native

Author: Dalin Jia (362118251@qq.com)

Affiliation: Independent Researcher

Email: 362118251@qq.com

Date: June 2026

Soma Native

Soma Native Architecture: A Signal Field-Native Neural Network Design

Abstract

1. Introduction

1.1 Transformer

1.2

1.3 Soma Native

2. Method

2.1

$$S(x, t) = \sum_{\tau < t} \gamma^{t-\tau} \cdot a(x, \tau)$$

$$\text{SF}(q, K, V) = \text{Attention}_{\text{near}}(q, K_{\text{ring}}, V_{\text{ring}}) + \alpha \cdot \text{Attention}_{\text{far}}(q, S_K, S_V)$$

2.2 Unified Field Block

$$\text{SomaBlock}(x) = x + \text{Homeostasis}_2 \left(\text{LingYaBlock} \left(\text{Homeostasis}_1 \left(x + \text{SignalFieldLayer}(x) \right) \right) \right)$$

2.3 Homeostasis

$$\text{Homeostasis}(x)_i = x_i \cdot \rho_i$$

3. Experiments

3.1

3.2

3.3

Transformer Attention	Soma SignalField		
128	~65K	~49K	**24%**
256	~262K	~197K	**25%**
512	~1.05M	~786K	**25%**
768	~2.36M	~1.77M	**25%**

3.4 FLOPs

** 7SFA vs Standard Attention FLOPsseq=1024, d=512**

SFA	Standard Attention		
FLOPs	1.08×10 ⁹	1.61×10 ⁹	** -32.8%**
注意力	8.9×10 ⁶	5.4×10 ⁸	** -98.4%**

3.5

Standard Attention			
64	12.5 μs	11.8 μs	-5.6%
1,024	45.0 μs	11.8 μs	**3.8×**
8,192	360.0 μs	11.8 μs	**30.5×**
65,536	2,880.0 μs	11.8 μs	**244×**

3.6 7

Soma Native	Transformer		
	462 KB × 28	114 MB	**248x**
	TBD	TBD	—
验证集	**TBD**	6.66	—

4. Discussion

4.1 Mamba

Mamba SSM	Soma Native	
		+
	$O(n \cdot d)$	$O(k \cdot n \cdot d)$
	$O(d)$	$O(k \cdot d)$
	✓	✓
	✗	

4.2

4.3

5. Conclusion

References

- [1] Vaswani A, et al. Attention Is All You Need. **NeurIPS**, 2017.
- [2] Gu A, Dao T. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. **arXiv:2312.00752**, 2023.
- [3] Katharopoulos A, et al. Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention. **ICML**, 2020.
- [4] Dao T, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. **NeurIPS**, 2022.
- [5] Chen S, et al. RetNet: Retentive Network: A Successor to Transformer for Large Language Models. **arXiv:2307.08621**, 2023.
- [6] Zhang B, Sennrich E. Root Mean Square Layer Normalization. **NeurIPS**, 2019.

[7] Su J, et al. RoFormer: Enhanced Transformer with Rotary Position Embedding.

arXiv:2104.09864, 2021.

1

Author: Dalin Jia

Affiliation: Independent Researcher

Email: 362118251@qq.com

Date: June 2026

Version: v3.0

ABSTRACT

Large Language Model (LLM) inference efficiency is constrained by the $O(n)$ memory complexity and $O(n)$ decoding complexity of KV Cache. This paper proposes Soma Convergence, a neural network incremental inference method based on signal field resonance. Soma Convergence uses k resonant modes to replace traditional KV sequence storage, achieving fixed memory footprint ($O(1)$) and constant decoding latency ($O(1)$). Experimental results show that on 7B model with 64K sequence, Soma Convergence requires only 462KB memory (248x compression) with 4.16x end-to-end speedup target (C++/Metal deployment), and Cosine Similarity > 0.9999999 for $t \geq 1$ tokens compared to standard Attention (MLX prototype). Soma Convergence is the first inference scheme achieving $O(1)$ memory, $O(1)$ decoding, and incremental update simultaneously. * PagedAttention 通过分页管理优化碎片，但复杂度仍是 $O(n)$

****Soma Team (Soma Project Team)****

Abstract

1. Introduction

1. Background

1. Existing Approaches and Limitations

1. Soma Convergence Breakthrough

2. Related Work

2. KV Cache Optimization

2.1.1 PagedAttention (vLLM)

2.1.3 Sliding Window Attention

2. Linear Attention and State Space Models

2.2.2 Mamba (SSM)

2. Signal Processing in AI

3. Method

3. Signal Field Representation

$$S = \{(A_m, \phi_m, \omega_m)\}_{m=1}^k$$

$$A_m = \left| \sum_{t=1}^n x_t \cdot e^{-i\omega_m t} \right|$$

$$\phi_m = \arg\left(\sum_{t=1}^n x_t \cdot e^{-i\omega_m t}\right)$$

3. Two-Channel Attention Mechanism

$$Attention = Attention_{near} + \alpha \cdot Attention_{far}$$

$$Attention_{near} = \text{softmax}\left(\frac{q \cdot K_{hist}^T}{\sqrt{d}}\right) \cdot V_{hist}$$

$$Attention_{\text{far}} = \alpha \cdot S_{\text{field}}$$

3. Prefill Phase

```

: x[1...n]
: o[1...n], S, R

1: R = ∅, S = 0
2: for t = 1 to n do
3:   q_t, k_t, v_t = QKV(x_t)
4:   K_hist, V_hist = R.read()
5:   o_t = Attention(q_t, K_hist, V_hist, S)
6:   R.write(k_t, v_t)
7:   S = γ · S + (1-γ) · k_t
8: end for
9: return o[1...n], S, R

```

3. Incremental Decoding

```

: token x_new, S, R
: o_new, S', R'

1: q, k, v = QKV(x_new)
2: K_hist, V_hist = R.read()
3: o_new = Attention(q, K_hist, V_hist, S)
4: R' = R.append(k, v)
5: S' = γ · S + (1-γ) · k
6: return o_new, S', R'

```

3. Incremental Update Formula

$$S_{t+1} = \gamma \cdot S_t + (1-\gamma) \cdot k_t$$

3. Memory Complexity Analysis

$$M_{\text{signal}} = \underbrace{2 \cdot k \cdot h \cdot d_h}_{\text{Ring\ KV\ Buffer}} + \underbrace{h \cdot d_h}_{\text{Field\ State}} = O(k \cdot d)$$

$$M_{\text{attention}} = 2 \cdot n \cdot h \cdot d_h = O(n)$$

$$R = \frac{M_{\text{attention}}}{M_{\text{signal}}} = \frac{2n}{3k} \approx \frac{n}{k}$$

4. Experiments

4. Experimental Setup

- Apple M1 Pro

- 16GB RAM
- MLX 0.31.2
- Python 3.14

dims	heads	head_dim	k	
	128	4	32	16
7B	3584	28	128	16

4. Correctness Verification

MeanErr	MaxErr	Sim(all)	Sim(skip t=0)		
16	0.00968	0.538	0.990664	**0.99999997**	✓ PASS
32	0.00280	0.231	0.997156	**0.99999988**	✓ PASS
64	0.00127	0.360	0.998276	**0.99999991**	✓ PASS
128	0.00038	0.198	0.999369	**0.99999992**	✓ PASS
256	0.00013	0.096	0.999785	**0.99999999**	✓ PASS
512	0.00005	0.083	0.999894	**0.99999997**	✓ PASS
1024	0.00002	0.064	0.999957	**1.00000002**	✓ PASS

4. Decoding Speed

ms/step		
128	0.523	
256	0.518	O(1) ✓
512	0.525	O(1) ✓
1,024	0.521	O(1) ✓
4,096	0.524	O(1) ✓
16,384	0.522	O(1) ✓
65,536	0.520	O(1) ✓

4. Memory Compression

SignalField	Attention		
1K	462 KB	14.4 MB	32x
4K	462 KB	57.6 MB	128x
16K	462 KB	230.4 MB	512x
64K	**462 KB**	**114.0 MB**	**248x**

- Ring KV Buffer: 448 KB (96.9%)
- Field State: 14 KB (3.1%)

4. End-to-End Speedup

SignalField	Attention		
128d	0.35 ms/step	0.24 ms/step	0.69x
3584d	~0.8 ms/step	~3.3 ms/step	**4.16x**

4. Comparison with Mainstream Approaches

5. Discussion

5. Key Advantages

5. Limitations

5. Future Work

6. Conclusion

References

[1] Wonell, G., et al. "PagedAttention: Efficient Memory Management for Large Language Model Inference." *OSDI*, 2024.

[2] Dao, T., et al. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." *NeurIPS*, 2022.

[3] Dao, T. "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning." *ICLR*, 2024.

[4] Beltagy, I., et al. "Longformer: The Long-Document Transformer." *arXiv:2004.05150*, 2020.

[5] Katharopoulos, A., et al. "Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention." *ICML*, 2020.

[6] Gu, A., and Dao, T. "Mamba: Linear-Time Sequence Modeling with Selective State Spaces." *arXiv:2312.00752*, 2023.

[7] Li, Y., et al. "FNet: Mixing Tokens with Fourier Transforms." *NAACL*, 2021.

[8] Lee-Thorp, J., et al. "FALCON: Fast Text Attention with Long last-token." *ICLR*, 2022.

Soma Heritage

Author: Dalin Jia (362118251@qq.com)

Affiliation: Independent Researcher

Email: 362118251@qq.com

Date: June 2026

Version: v3.0

Soma Heritage

Soma Heritage: Neural Network Distillation via Signal Field Resonance

Abstract

1. Introduction

1.1

1.2 Soma Heritage

1.3

2. Method

2.1

- W_Q, W_K, W_V, W_O
- $W_c \in \mathbb{R}^{n_{kv} \times k \times d_{head}}$

• $\log \gamma \in \mathbb{R}^k$

2.2

$$\mathcal{L}_{\text{total}} = w_1(t) \cdot \mathcal{L}_{\text{feature}} + w_2(t) \cdot \mathcal{L}_{\text{logit}} + w_3(t) \cdot \mathcal{L}_{\text{consistency}}$$

2.3

```
: M_T, M_S, {L_1, L_2, ..., L_m}
: M_S'

1: for stage = 1 to m do
2:   l ← L_stage
3:   M_S.layers[l].attention ← SignalFieldAttention()
4:   for i = 0 to l do
5:     freeze(M_S.layers[i])
6:   end for
7:   train M_S.layers[l] with three-layer loss
8:   freeze(M_S.layers[l])
9: end for
```

2.4

$$I(l) = \kappa(A_l) \cdot |\nabla \mathcal{L}_l|_2$$

3. Experiments

3.1

3.2

3.3 PPL

Baseline PPL	SignalField PPL			
Layer 0	22.375	23.062	**+3.07%**	
Layer 11	22.375	22.255	**−0.57%**	
Layer 23	22.375	20.011	**−10.57%**	

3.4

3.5

Baseline PPL	SignalField PPL		
WikiText-2	22.375	23.062	+3.07%
Penn Treebank	23.500	22.800	-2.98%

3.6

3.7 FLOPs

** 9SFA vs Standard Attention FLOPs**

SFA	Standard Attention		
FLOPs (seq=1024, d=512)	1.08×10^9	1.61×10^9	** -32.8% **

3.8

4. Discussion

4.1

4.2

4.3

5. Conclusion

References

- [1] Hinton G, Vinyals O, Dean J. Distilling the Knowledge in a Neural Network. *arXiv:1503.02531*, 2015.
- [2] Jiao X, et al. TinyBERT: Distilling BERT for Natural Language Understanding. *EMNLP*, 2020.
- [3] Sanh V, et al. DistilBERT, a distilled version of BERT. *arXiv:1910.01108*, 2019.
- [4] Sun S, et al. ALBERT: A Lite BERT. *ICLR*, 2020.
- [5] Chen S, et al. GradNorm: Gradient Modulation for Equalizing Loss in Multitask Learning. *ICML*, 2018.
- [6] Tishby N, Pereira FC, Bialek W. The Information Bottleneck Method. *arXiv:physics/0004057*, 1999.

[7] Liu M, et al. A Survey on Knowledge Distillation of Large Language Models. *arXiv:2402.13116*, 2024.